

CS 3081 · ARTIFICIAL INTELLIGENCE · FALL 2026

Assignment 1 — Search Strategies

BFS · DFS · Greedy Best-First · A*

Name: Juri Alamri**ID:** S21107373

Dr Naila Marir

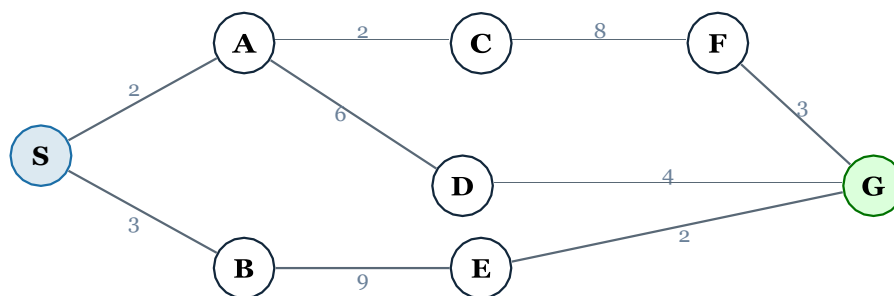
Instructor: Dr Naila Marir**Total marks:** 100**Sections:** 3081-1 (CRN 10480), 3081-2 (CRN 10481)

Instructions. Answer all four questions. Show your working — a correct final answer with no reasoning earns partial credit only. Where a question asks for the order in which nodes are expanded, write the nodes as a sequence, e.g. *S, A, C, G*.

- When two nodes have equal priority, break ties **alphabetically**.
- A node is *expanded* when it is removed from the frontier and its successors are generated. Counting the goal test, a node is expanded even if it turns out to be the goal.
- Assume the **graph-search** form of each algorithm: never expand the same state twice.
- This is an individual assignment. You may discuss ideas, but the work you submit must be your own.

1 Uninformed search: BFS and DFS

Consider the following state graph. The start state is *S* and the goal state is *G*. Edges are undirected, and the number on each edge is its step cost.



For this question ignore the edge costs. BFS and DFS are *uninformed*: they do not look at cost or at any estimate of distance to the goal.

(a) Breadth-first search**[8 marks]**

List the order in which states are expanded by BFS, and give the path it returns.

Frontier progression:

Start:
SAfter expanding S:
A, B, C

After expanding A:
B, C, D

After expanding B:
C, D, E

After expanding C:
D, E, F

After expanding D:
E, F, G

After expanding E:
F, G

After expanding F:
G

Order of expansion:

S, A, B, C, D, E, F, G

Path returned:

S -> A -> D -> G

(b) Depth-first search

[8 marks]

List the order in which states are expanded by DFS, and give the path it returns. Assume successors are pushed onto the stack in reverse alphabetical order, so that the alphabetically first successor is expanded next.

DFS uses a stack. Successors are pushed in reverse alphabetical order so that the alphabetically first successor is expanded first.

Stack progression:

Start:
S

Expand S:
A, B, C

Expand A:
D, B, C

Expand D:
G, B, C

Expand G: Goal reached

Order of expansion: S, A, D, G

Path returned: S -> A -> D -> G

(c) Comparison**[6 marks]**

BFS and DFS returned different paths. State which path is shorter *in number of steps*, and explain why BFS is guaranteed to find that path while DFS is not.

The shorter path in number of steps is:

S → A → D → G

BFS is guaranteed to find the shortest path in terms of number of steps because it expands nodes level by level. DFS is not guaranteed to find the shortest path because it may go deep into a longer path before exploring other possibilities.

(d) Frontier data structures**[3 marks]**

BFS and DFS differ by one design choice. Name the data structure each uses for its frontier, and explain in one sentence how that choice produces the difference in behaviour you observed.

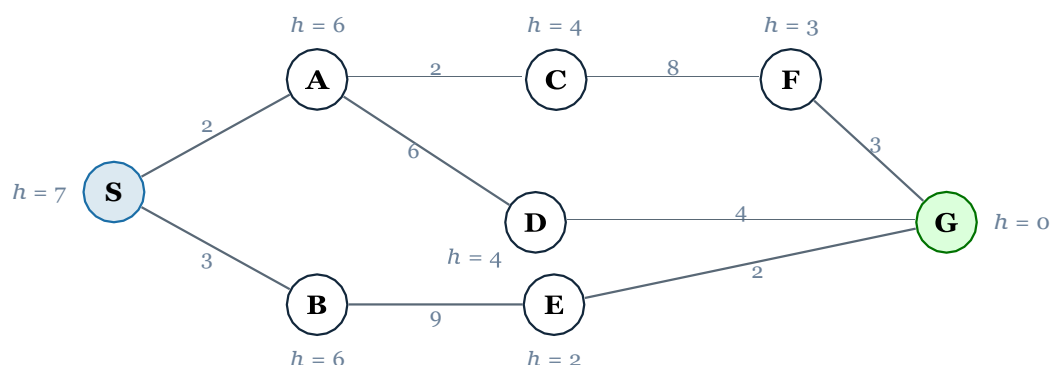
BFS uses a Queue (FIFO).

DFS uses a Stack (LIFO).

The different frontier structures cause BFS to explore level by level, while DFS explores one branch deeply before backtracking.

2 Informed search: Greedy Best-First and A*

The same graph is repeated below. Each state is now annotated with a heuristic value $h(n)$ — an estimate of the cost from n to the goal G .

**(a) Greedy best-first search****[8 marks]**

Greedy best-first expands the frontier node with the smallest $h(n)$. List the order of expansion and the path returned.

Greedy Best-First Search selects the node with the smallest $h(n)$.

Heuristic values:

S = 7

A = 6

B = 6

C = 4

D = 4

E = 2

F = 3

G = 0

Expansion steps:

Start:

S

Expand S:

Frontier = A(6), B(6), C(4)

Expand C:

Frontier = A(6), B(6), F(3)

Expand F:

Frontier = A(6), B(6), G(0)

Expand G:

Goal reached

Order of expansion:

S, C, F, G

Path returned:

S -> C -> F -> G

(b) A* search**[12 marks]**

A* expands the frontier node with the smallest $f(n) = g(n) + h(n)$. Complete the table for every node *at the moment it is expanded*, then give the order of expansion and the path returned.

A* uses:

$$f(n) = g(n) + h(n)$$

Expansion table:

Order	Node	$g(n)$	$h(n)$	$f(n)=g(n)+h(n)$
1	S	0	7	7
2	C	2	4	6
3	A	2	6	8
4	B	3	6	9
5	F	6	3	9
6	G	9	0	9

Order of expansion: S, C, A, B, F, G

Path returned: S -> C -> F -> G

Total cost: 9

(c) Why they differ**[6 marks]**

Greedy best-first and A* returned different paths, or explored a different number of nodes, or both. Explain the difference by referring to what each algorithm ignores.

Greedy Best-First Search only considers $h(n)$, so it chooses nodes that appear closest to the goal without considering the cost already spent.

A* considers both the cost so far $g(n)$ and the estimated remaining cost $h(n)$ using $f(n) = g(n) + h(n)$, so it can avoid expensive paths.

3 Admissibility and optimality**(a) Definition****[5 marks]**

Define an **admissible** heuristic. Your definition must be precise enough that someone could check a given heuristic against it.

An admissible heuristic is a heuristic that never overestimates the true cost to reach the goal.

For every node n :

$$h(n) \leq h^*(n)$$

where $h^*(n)$ is the actual cheapest cost from n to the goal.

(b) Testing the heuristic

[10 marks]

For the graph in Question 2, compute the true cost $h^*(n)$ of the cheapest path from each node to G , and decide whether the given h is admissible.

True cost $h^*(n)$ from each node to G :

Node	S	A	B	C	D	E	F	G
$h(n)$ given	7	6	6	4	4	2	3	0
$h^*(n)$ true	8	6	5	5	3	2	3	0
$h \leq h^*$?	Yes	Yes	No	Yes	No	Yes	Yes	Yes

Is h admissible? No, the heuristic is not admissible because it overestimates the true cost at some nodes.

If not, name every node that violates the condition: B and D

(c) Consequence**[6 marks]**

Suppose a heuristic overestimates the true cost at exactly one node. Explain what this can do to the solution A^* returns, and why BFS could never suffer from this problem.

If a heuristic overestimates the true cost at one node, A^* may choose a more expensive path and lose its guarantee of finding the optimal solution.

BFS cannot suffer from this problem because it does not use a heuristic; it explores nodes based only on depth.

(d) A trivial heuristic**[3 marks]**

What does A^* reduce to when $h(n) = 0$ for every node? Justify your answer in one sentence.

When $h(n) = 0$ for every node, A^* becomes Uniform Cost Search (UCS).

This is because:

$$f(n) = g(n) + h(n)$$

and $h(n) = 0$, so:

$$f(n) = g(n)$$

Therefore A^* selects nodes only according to the path cost.

4 Comparing the four strategies

(a) Properties table

[8 marks]

Complete the table. Name the data structure each strategy uses for its frontier. For *optimal*, write Yes or No and state any condition required for that answer to hold.

Strategy	Frontier	Optimal?
Breadth-first	Queue (FIFO)	Yes, when all step costs are equal
Depth-first	Stack (LIFO)	No
Greedy best-first	Priority Queue (based on $h(n)$)	No
A*	Priority Queue (based on $f(n)=g(n)+h(n)$)	Yes, if the heuristic is admissible

(b) Choosing a strategy

[13 marks]

For each scenario, name the most appropriate of the four strategies and justify your choice. Refer to the properties in part (a) — a bare name earns no marks.

- i. A robot must find *any* route out of a building during a fire. Memory is very limited; the route need not be shortest.

Strategy: Depth-first search (DFS)

DFS uses a stack and requires less memory than BFS because it stores mainly the current path. It can find a route without needing the shortest path.

- ii. A route planner must return the *cheapest* driving route, and a good straight-line-distance estimate is available.

Strategy: A*

A* uses both the cost already traveled $g(n)$ and the estimated remaining cost $h(n)$. With an admissible heuristic, it can find the cheapest route.

- iii. A puzzle has uniform step costs, no useful heuristic, and the solution is known to be shallow.

Strategy: Breadth-first search (BFS)

BFS is complete and optimal when step costs are equal. It is suitable when the solution is shallow and no heuristic information is available.

(c) Short answer**[4 marks]**

A* is optimal but is often impractical on large problems. State the reason, and name one property of the search that makes it worse.

A* is often impractical on large problems because it requires a large amount of memory to store generated nodes in the frontier.

The property that makes it worse is that A* has high memory usage because it keeps many explored and unexplored nodes.
